Практическая работа №3 Объекты и классы: абстрактные классы и интерфейсы, пакеты.

Кармаин Андрей Александрович · группа ИНБ-30-25 · 21.09.2026

Исходники: github.com/akarmain/vanilla_code/tree/main/MIREA/trem_2/Java/Workbook_3

Часть 1. Задача №1: абстрактный Vehicle, интерфейс ElectricVehicle, классы Car и ElectricCar, пакеты vehicles/app

0_vehicle_и_интерфейс_electricvehicle.java

```
// Часть 1. Задача №1: абстрактный Vehicle, интерфейс ElectricVehicle, классы Car и ElectricCar, пакеты vehicles/app
public class TestCar {  // это app.TestCar
    public static void main(String[] args) {
        Car car = new Car("Lada Vesta", "A123BC777", "белый", 2021);
        ElectricCar tesla = new ElectricCar("Tesla Model 3", "E777ЛК77", "красный", 2023, 75.0);

        car.setYear(2022);                          // изменение года выпуска
        car.setOwnerName("Иванов И.И.");            // изменение владельца
        tesla.setOwnerName("Петров П.П.");
        tesla.setInsuranceNumber("ОСАГО-112233");   // изменение страхового номера

        System.out.println("Ёмкость батареи: " + tesla.getBatteryCapacity() + " кВт⋅ч");

        System.out.println(car);
        System.out.println(tesla);
    }
}

abstract class Vehicle {
    private String model;
    private String license;
    private String color;
    private int year;
    private String ownerName;
    private String insuranceNumber;
    protected String engineType;

    public Vehicle(String model, String license, String color, int year) {
        this.model = model;
        this.license = license;
        this.color = color;
        this.year = year;
    }

    public abstract String vehicleType();

    public String getModel() { return model; }
    public void setModel(String model) { this.model = model; }

    public String getLicense() { return license; }
    public void setLicense(String license) { this.license = license; }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }

    public int getYear() { return year; }
    public void setYear(int year) { this.year = year; }

    public String getOwnerName() { return ownerName; }
    public void setOwnerName(String ownerName) { this.ownerName = ownerName; }

    public String getInsuranceNumber() { return insuranceNumber; }
    public void setInsuranceNumber(String insuranceNumber) { this.insuranceNumber = insuranceNumber; }
```

```
    public String getEngineType() { return engineType; }
    public void setEngineType(String engineType) { this.engineType = engineType; }

    @Override
    public String toString() {
        return vehicleType() + ": " + model + " (" + license + "), " + color + ", " + year
             + ", владелец " + ownerName + ", страховка " + insuranceNumber
             + ", двигатель " + engineType;
    }
}

interface ElectricVehicle {          // интерфейс = список методов без реализации
    double getBatteryCapacity();
    void setBatteryCapacity(double batteryCapacity);
}

class Car extends Vehicle {
    public Car(String model, String license, String color, int year) {
        super(model, license, color, year);
        this.engineType = "Combustion";
    }

    @Override
    public String vehicleType() { return "Car"; }
}

class ElectricCar extends Car implements ElectricVehicle {  // наследует класс + реализует интерфейс
    private double batteryCapacity;

    public ElectricCar(String model, String license, String color, int year, double batteryCapacity) {
        super(model, license, color, year);
        this.batteryCapacity = batteryCapacity;
        this.engineType = "Electric";
    }

    @Override
    public double getBatteryCapacity() { return batteryCapacity; }

    @Override
    public void setBatteryCapacity(double batteryCapacity) { this.batteryCapacity = batteryCapacity; }

    @Override
    public String vehicleType() { return "Electric Car"; }

    @Override
    public String toString() {
        return super.toString() + ", батарея " + batteryCapacity + " кВт*ч";
    }
}
```

Входные данные: не требуется

Выходные данные:

```
Ёмкость батареи: 75.0 кВт*ч
Car: Lada Vesta (A123BC777), белый, 2022, владелец Иванов И.И., страховка null, двигатель Combustion
Electric Car: Tesla Model 3 (E777ЛК77), красный, 2023, владелец Петров П.П., страховка ОСАГО-112233,
двигатель Electric, батарея 75.0 кВт*ч
```

Часть 2. Задача №1: интерфейсы BikeParts, MountainParts (TERRAIN), RoadParts для проекта bikeproject

1_интерфейсы_велосипедного_проекта.java

```
// Часть 2. Задача №1: интерфейсы BikeParts, MountainParts (TERRAIN), RoadParts для проекта bikeproject
public class Bike {
    protected String size;
```

```
    protected String colour;
    protected int gears;

    public Bike(String size, String colour, int gears) {
        this.size = size;
        this.colour = colour;
        this.gears = gears;
    }

    public String describe() {
        return BikeParts.MANUFACTURER + ": размер " + size + ", цвет " + colour + ", передач " + gears;
    }

    public static void main(String[] args) {
        MountainBike mtb = new MountainBike("L", "чёрный", 21, "воздушная вилка", "хардтейл");
        RoadBike road = new RoadBike("M", "красный", 18, "25 мм", "72 см");

        System.out.println(mtb.describe());
        System.out.println("Покрытие: " + MountainParts.TERRAIN
                           + ", подвеска: " + mtb.getSuspension() + ", тип: " + mtb.getType());

        System.out.println(road.describe());
        System.out.println("Покрытие: " + RoadParts.TERRAIN
                           + ", ширина покрышки: " + road.getTyreWidth()
                           + ", высота штыря: " + road.getPostHeight());
    }
}

interface BikeParts {
    String MANUFACTURER = "BikeCorp";  // в интерфейсе поле само по себе public static final
    String describe();
}

interface MountainParts extends BikeParts {
    String TERRAIN = "off_road";

    void setSuspension(String newValue);
    String getSuspension();
    void setType(String newValue);
    String getType();
}

interface RoadParts extends BikeParts {
    String TERRAIN = "track_racing";

    void setTyreWidth(String newValue);
    String getTyreWidth();
    void setPostHeight(String newValue);
    String getPostHeight();
}

class MountainBike extends Bike implements MountainParts {
    private String suspension;
    private String type;

    public MountainBike(String size, String colour, int gears, String suspension, String type) {
        super(size, colour, gears);
        this.suspension = suspension;
        this.type = type;
    }

    @Override public void setSuspension(String newValue) { this.suspension = newValue; }
    @Override public String getSuspension() { return suspension; }
    @Override public void setType(String newValue) { this.type = newValue; }
    @Override public String getType() { return type; }
}

class RoadBike extends Bike implements RoadParts {
    private String tyreWidth;
```

```
    private String postHeight;

    public RoadBike(String size, String colour, int gears, String tyreWidth, String postHeight) {
        super(size, colour, gears);
        this.tyreWidth = tyreWidth;
        this.postHeight = postHeight;
    }

    @Override public void setTyreWidth(String newValue) { this.tyreWidth = newValue; }
    @Override public String getTyreWidth() { return tyreWidth; }
    @Override public void setPostHeight(String newValue) { this.postHeight = newValue; }
    @Override public String getPostHeight() { return postHeight; }
}
```

Входные данные: не требуется

Выходные данные:

```
BikeCorp: размер L, цвет чёрный, передач 21
Покрытие: off_road, подвеска: воздушная вилка, тип: хардтейл
BikeCorp: размер M, цвет красный, передач 18
Покрытие: track_racing, ширина покрышки: 25 мм, высота штыря: 72 см
```